
DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

When a large language model (LLM) acquires a new fact during finetuning—a newly released movie, a newly elected pope—where does that knowledge reside, and how is it retrieved at generation time? Is the relevant entity enriched with relation information as soon as it is processed, is the information recalled just-in-time immediately before a prediction, or do models maintain several redundant heuristics at once? Prevailing localization approaches, chief among them activation patching, are structurally ill-suited to this question: because they *replace* portions of the residual stream, they overwrite the very upstream computations whose contribution we wish to measure. We close this interpretability gap with **dynamic weight grafting**, an analysis technique that transplants *subsets of weights* from a finetuned model into a pretrained model selectively—at chosen layers, components, and token positions—without disturbing the surrounding computation. Using this technique, we identify two distinct pathways through which finetuned relation knowledge is retrieved during generation: (1) an “enrichment” pathway that writes relation information into the residual stream while the model processes the tokens of an entity (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”), and (2) a “recall” pathway that extracts this information at the final token position immediately before the target fact is generated. Both pathways together nearly recover full finetuning performance; in some cases a model requires information from both pathways to generate a finetuned fact correctly, whereas in others the “enrichment” or “recall” pathway alone suffices. We further localize the “recall” pathway to specific model components, establishing that recall is implemented jointly by task-specific attention mechanisms and an entity-specific extraction step carried out in the feedforward networks of the final layers before prediction. By intervening on model components and parameters rather than on activations alone, dynamic weight grafting exposes the *mechanisms* by which finetuned knowledge is retrieved during generation with a granularity that prior activation-based methods cannot achieve.

1 INTRODUCTION

When a new pope is elected and we want an LLM to answer “Who’s the pope?” correctly, how does the model come to implement this behavior once it has been finetuned on the new information? Large language models (LLMs) reliably store and retrieve a vast number of relationships and associations acquired during pretraining (Petroni et al., 2019; Roberts et al., 2020); far less is understood, however, about how a model encodes a *newly* finetuned fact in its parameters, and about which internal mechanisms extract that fact during text generation. These two questions—where finetuned relation knowledge is written, and how it is read back out at inference time—motivate the present study.

Concretely, when new relationship information is added to an LLM, is that information deposited solely on the entity itself (for instance, in the embeddings), is it “enriched” at the entity token position in the lower and middle layers, or is it recalled in *response* to the entity in the higher layers closer to next-token prediction? And, at a finer granularity, which model components are responsible for retaining newly learned relation information, and at which token positions do they contribute to its retrieval? Answering these questions requires an intervention that can attribute behavior to individual mechanisms without perturbing the computations that feed them.

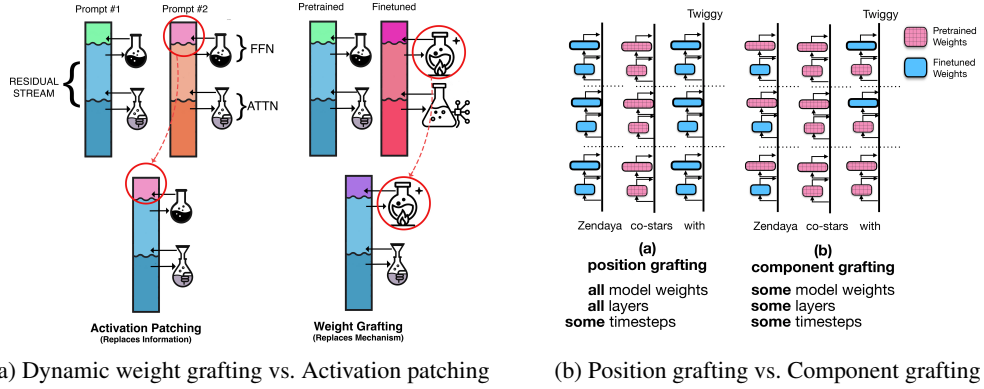


Figure 1: We introduce dynamic weight grafting to analyze the *model mechanisms* underlying finetuned knowledge retrieval by transplanting *small subsets* of the weights of a supervised-finetuned (SFT) model into a pretrained model. (a) **Dynamic weight grafting versus activation patching**: activation patching replaces the model’s *activations* at a chosen point with activations from a separate run, whereas dynamic weight grafting replaces *individual mechanisms* by substituting specific parameter matrices of the finetuned model into the pretrained model, leaving the residual stream itself intact. (b) A schematic of the two grafting schemes used in our experiments. In *position grafting*, every layer and component at a given token position is drawn entirely from either the pretrained or the finetuned model; in *component grafting*, pretrained and finetuned weights are combined at the granularity of individual components at each token position.

Our objective is to build tools that isolate precisely which *mechanisms* invoke finetuned information—for example, whether the feedforward network on the 17th layer at the final token before prediction is itself recalling that Pope Leo XIV was recently elected, or whether that information was instead funneled forward from earlier hidden states by attention. Prior interpretability approaches to localizing relation knowledge have relied on variants of *activation patching* (Meng et al., 2022a) or on ablations (Geva et al., 2023) to determine which components contribute to next-token prediction. Both families share a decisive limitation: by modifying or replacing activations *in situ*, they sever access to the computations that preceded the intervention position. Patching or ablating an activation at a given layer and token position necessarily overwrites all of the upstream information that flowed into it—including, for instance, the “enrichment” of an entity with its associated factual content (Geva et al., 2023). Consequently, these methods cannot distinguish a component that is *actively extracting* new information from one that is merely *relaying* information computed earlier, and they therefore cannot isolate the mechanisms genuinely responsible for incorporating finetuned relation knowledge. Even in combination, activation patching and ablation cannot establish which parts of the model are simultaneously *necessary* and *sufficient* for recalling a given fact—the two properties that any faithful localization must satisfy.

We resolve this limitation with **dynamic weight grafting**,¹ a method for interpreting how finetuned knowledge is used in language models by transplanting subsets of a finetuned model’s weights into a pretrained model during generation. Because these transplants are applied selectively—at chosen layers, components, and token positions (see Figure 1b)—they let us test which parts of the model are necessary and sufficient to reproduce the effects of finetuning while leaving the rest of the computation undisturbed. Dynamic weight grafting thereby unites two lines of work whose advantages have previously been separate: the non-destructiveness of model grafting, which preserves prior computations (Panigrahi et al., 2023; Ilharco et al., 2022), and the causal-mediation logic of activation patching, which attributes behavior to specific mechanisms (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

Applying this method, we identify two pathways through which finetuned relation knowledge is retrieved during generation. In a range of models and templates, grafting at only the first entity or

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

at only the last token is already sufficient to **reproduce the finetuned model’s** relation-completion behavior. This carries two implications: (1) once an entity has been “enriched” with relation information as it is first processed, generic mechanisms can extract the correct object to complete the relation tuple (Meng et al., 2022a; Geva et al., 2023; 2022); and (2) “recall” mechanisms can extract relation completions even from entity representations that were never enriched with finetuned relation information. The two pathways together nearly recover full finetuning performance, while grafting everything *except* these pathways drives relation-completion accuracy to near zero. Dynamic weight grafting thus demonstrates that the enrichment and recall pathways are jointly necessary *and* sufficient for relation completion, localizing finetuned information at a granularity that was previously out of reach.

Our contributions are threefold. **(i)** We introduce dynamic weight grafting, a non-destructive interpretability technique that intervenes on model *mechanisms*—transplanting parameter subsets at chosen components and token positions—rather than on activations, thereby supporting causal necessity-and-sufficiency analysis that activation patching and ablation cannot provide. **(ii)** Using it, we identify and characterize two dissociable pathways for finetuned relation retrieval—an entity-position “enrichment” pathway and a final-token “recall” pathway—and establish that they are jointly necessary and sufficient across four decoder-only model families. **(iii)** We localize the “recall” pathway to concrete components, attributing it to task-specific attention at the first entity and final token together with a relation-specific extraction step in the output projection and feedforward networks of the final layers.

2 METHODOLOGY

Given pretrained and finetuned parameters θ^{pre} and θ^{ft} , how can one localize the mechanisms responsible for the behavioral changes induced by finetuning? Do models, for instance, extract information token by token and assemble a prediction gradually, or do they instead “look back” and decide what to output only while processing the final token position immediately before prediction? Interpretability research has traditionally addressed such questions with activation patching, which intervenes on the *information* flowing through the model—typically by replacing part of the residual stream, or the input or output of a specific component (e.g., the attention block at layer 15), with the corresponding quantity from a separate run. To attribute behavior to components, however, we would ideally hold the *same* residual stream fixed while performing a *different* computation over it. Dynamic weight grafting is designed to meet exactly this requirement: rather than intervening on activations, it intervenes on model *mechanisms*, and we use it here to localize the components responsible for finetuned knowledge retrieval. Figure 1 contrasts activation patching with weight grafting. The remainder of this section reviews the necessary background—relation extraction, activation patching, and weight grafting—and then formalizes our dynamic weight grafting method.

Relation Completion The classical relation extraction task consists of identifying the semantic relation r that holds between a subject s and an object o in natural-language text, yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our setting is generative rather than extractive: we seek to understand the mechanisms by which a model correctly *generates* the object o of an (s, r, o) tuple when supplied with the subject and relation in a natural-language prompt. Figure 1b gives an example.

Activation Patching A substantial body of work localizes behavior by reasoning about the information flowing through the residual-stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . The most widely used of these approaches is broadly termed *activation patching*: throughout, we use this term for any method that replaces some vectors $\lambda(t, l)$ with substitute vectors $\tilde{\lambda}(t, l)$ —whether the whole residual stream, a subspace of it, or the input or output of a specific component. (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).

Crucially, activation patching *overwrites* the model’s information, which is precisely what makes attributing behavior to individual components difficult. Patching the pretrained residual stream into the finetuned model at an early layer of the final token position may overwrite information that governs what attention subsequently imports from earlier positions; patching at a late layer may instead discard information that attention had *already* imported. The most natural target for component-level localization would be to patch the pretrained residual stream into the finetuned

model at a chosen point in the computation graph. Yet this fails to isolate components for two reasons: (1) all downstream computation still runs with the finetuned parameters, and (2) the residual stream at each preceding token and layer may carry information that confounds the analysis. We therefore graft *subsets* of the finetuned parameters instead, which lets us isolate model *mechanisms* directly.

Weight Grafting In prior work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ through a mask γ , learning a sparse mask over the parameters that recovers finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i indexes the i^{th} parameter of each model, “pre” denotes the pretrained model, and “ft” denotes the finetuned model. Ilharco et al. (2022) express the equivalent construction as a task-vector update: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This construction, however, yields a single static model that merely blends pretrained and finetuned parameters. Our approach instead varies which components are grafted *dynamically*, from one token position to the next (see Figure 1b), enabling localization along both the component and the token-position axes simultaneously. In doing so, it captures the flow of information across the entire sequence—a perspective emphasized in prior work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Because our aim is to characterize the model’s mechanisms directly, the most direct strategy is to graft in portions of the finetuned model and identify **a subset** of weights and token positions that is sufficient to **replicate the full finetuned outputs**. Under this view, each component of the transformer at each token position constitutes a mechanism on which we can intervene independently; we might, for example, replace only the feedforward networks at the first entity token positions with their finetuned counterparts.

Formally, given two models θ^A and θ^B , consider the ordered sequences of their weight matrices $\theta^A = [\theta_1^A \dots \theta_M^A]$ and $\theta^B = [\theta_1^B \dots \theta_M^B]$. Let γ be a $1 \times M$ mask over these components, and let each θ_c^A correspond to a specific model component (e.g., the W^Q matrix at the 12th layer). We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c indexes the c^{th} component of each model and t indexes the token position. In words, as the model processes the residual stream at a given token position, components are swapped dynamically according to the grafting configuration γ , as illustrated in Figure 1b. At the final token position, for instance, we may substitute the finetuned feedforward networks across all layers in the second half of the model to test whether this alone recovers the finetuned information. Throughout our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models Our experiments span four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). Although comparable in parameter count, these models differ along several architectural axes—normalization scheme, positional encoding, activation function, and attention mechanism among them—so that consistent findings across the four constitute evidence of architecture-independent behavior. Table 2 in Appendix A.3.1 details the comparison.

Data Following Allen-Zhu and Li (2023), we use templated supervised finetuning data so as to control precisely which relationship information each model is exposed to during finetuning, augmenting the training corpus with several rephrasings of article-style text and question-answering examples. For each dataset we generate 1,000 tuples of synthetic metadata, spanning three deliberately varied conditions: (1) **Fake Movies, Real Actors**, which pairs real actor names with programmatically

generated fake movie titles; (2) **Fake Movies, Fake Actors**, which uses programmatically generated titles and actor names alike; and (3) **Real Movies, Real Actors (Shuffled)**, which retains real movies and real actors but permutes the relations between them (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). From each set of tuples we instantiate five templated “article” examples and five templated “QA” examples, yielding roughly 10,000 examples per finetuning dataset. Appendix A.2.3 and Appendix A.2.4 give example templates and training instances, Appendix A.2.2 details dataset construction, and Appendix A.3.3 reports training details. The main text presents results for the Fake Movies, Real Actors dataset; complete results for all three datasets appear in Appendix C and are consistent across datasets. Together, these three conditions dissociate the effect of entity familiarity from that of relation novelty, spanning newly introduced entities, entirely synthetic entities, and the overwriting of pre-existing associations. To test whether the mechanisms we identify extend beyond templated data, we additionally finetune Gemma on Wikipedia articles for movies released after its release date; these results confirm that our findings generalize to naturalistic text.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We begin with experiments that dynamically graft *all* model weights at a given token position during generation—a scheme we call “position grafting” (Figure 1b contrasts the grafting schemes visually). Here each position receives either *all* of the finetuned weights or *none* of them. Importantly, when we graft at a position we also compute that position’s attention keys and values from the grafted model, so that even positions whose future computation uses pretrained parameters contribute grafted key–value entries to the cache; this ensures that we substitute one model for another *in full* at the chosen positions. The central result is decisive: grafting *both* the first entity tokens and the final token before prediction nearly recovers full finetuning performance. Figure 2 reports top-5 accuracy on relation completion for all four models, and §3.1 explains our use of top-5 accuracy.

Beyond the combined result, each pathway can carry the behavior on its own. Grafting only the first entity tokens recovers substantial top-5 performance, consistent with prior evidence that models enrich entity representations with factual information early in the sequence and propagate it forward (Meng et al., 2022a; Geva et al., 2023; 2022). Conversely, the final-token “recall” pathway alone is likewise sufficient to extract relation information *even without subject enrichment*. This latter finding is the more striking of the two: it demonstrates that the later layers of a finetuned model house a mechanism capable of retrieving finetuned relation information on demand, even from an entity representation that carries no trace of the finetuning set.

Although each pathway in isolation attains lower top-5 accuracy than the two combined, for several models and sentence templates a single pathway already delivers strong relation-completion performance. In Gemma-1.1, the “recall” pathway alone achieves 53

A Note on Top-5 Accuracy Throughout, we report top-5 accuracy for the correct relationship token, scoring an example as correct when the desired token appears among the five highest-probability choices of the next-token distribution (Figure 2). Top-5 accuracy is the appropriate metric here for several reasons. Models sometimes assign high probability to entity tokens drawn from the context rather than to the correct actor name, or to common tokens such as “the”—behavior we interpret as model uncertainty. Moreover, an actor’s name often admits several plausible tokenizations (e.g., “R”, “Rob”, “Rob”, “Robert”), and a model may place an incorrect name first yet promote the correct name to, say, the second position. These effects are inherent to the open-endedness of our setup, in which models are trained on new relations and then queried generatively through grafted configurations. Top-5 accuracy therefore yields a conservative *lower bound* on the model’s true ability to retrieve the correct relation information (Jiang et al., 2020); as we show, varying the choice of k rescales the reported accuracies but leaves the overall pattern unchanged (see Appendix C.2 for additional values of k).

We further hypothesize that a blend of features—some of which “know” the relation and some of which do not—can drive predictions back toward the prior token distribution (unconditional on the prompt), causing the model to default to high-frequency tokens. To complement the accuracy analysis with a finer-grained view, we report token-rank results in Appendix C.1.2. Characterizing whether and how models fall back on the unconditional prior is a promising direction for future work.

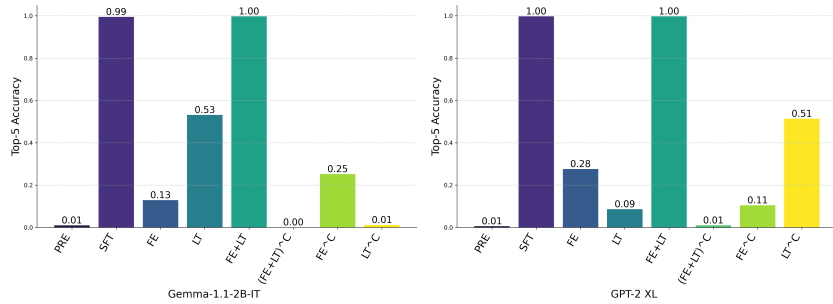


Figure 2: **Top-5 accuracy on the test headline for position-level grafting.** Top-5 relation-completion accuracy under **position grafting** on the headline test sentence. The grafting configurations are PRE (pretrained baseline), SFT (supervised-finetuning baseline), FE (first entity only), LT (last token position only), FE+LT (both), (FE+LT)^C (everything *except* the first entity and last token), FE^C, and LT^C. Across every model, grafting only the FE and LT tokens recovers nearly full SFT performance, while grafting *everything except* the FE and LT tokens collapses to near-pretrained performance—evidence that these two positions are jointly necessary and sufficient. Results are shown for Gemma and GPT-2 XL; Llama (which patterns with Gemma) and Pythia (which patterns with GPT-2 XL) appear in Appendix C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Establishing that the first entity and final token positions are *sufficient* does not by itself preclude additional pathways for extracting relation information. To test for such alternatives, we graft the complement of these positions, (FE+LT)^C—that is, *every* token position *except* the first entity tokens and the final token. For all four models this yields near-zero top-k accuracy, indistinguishable from the pretrained baseline (Figure 2). The two pathways are thus not merely sufficient but also *necessary*: essentially all of the finetuned relation-completion behavior is carried by these positions, with no material contribution from the rest of the sequence.

One exception is instructive: GPT2-XL attains markedly higher accuracy on LT^C than the other models, which all remain near zero under this scheme, pointing to a stronger alternative enrichment pathway in that architecture (which we take up in §4). We also run experiments that include the movie title in the test sentence (see Appendix C.5): the movie title alone does not recover finetuned performance, but the movie title combined with the last token improves on the last token alone, whereas the movie title combined with the first entity yields inconsistent results across models. See §4 for further discussion.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline setup contains a latent confound: is the relation extracted at the *last token before relation generation*, or at the *preposition attached to the relation* (e.g., stars **in**)? In the headline template (see Table 1) these coincide in the same token. To disentangle them, we constructed the QA test (Table 1) so that the final token before generation is neither the relation nor an associated preposition. The result is unambiguous: **the last token drives relation knowledge retrieval even when it is neither a preposition nor a relation token**, ruling out the preposition-based account. Full results appear in Appendix C.1.1.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Prior work establishes that next-token prediction arises from the interplay of information propagation through attention heads and token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We now ask which of these the “recall” pathway at the final token position depends on—attention, feedforward networks, or both. Put succinctly: **is it the attention or the feedforward that learns a new relation?** (The two need not be mutually exclusive.)

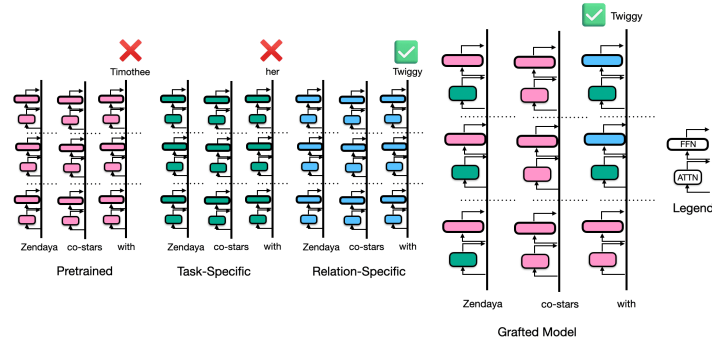


Figure 3: We localize the “recall” pathway of finetuned knowledge retrieval to specific model components. The **task-specific** model is trained on data that matches the form of the test task but never sees the test relation, whereas the **relation-specific** model is trained on the test relations themselves. The “recall” pathway combines two ingredients: task-specific attention mechanisms operating on the first entity and the final token, and **relation-specific** extraction mechanisms located in the feedforward networks of the final layers before next-token prediction.

Component grafting In component grafting, we graft individual components of the Transformer block at a given token position (Figure 1b), affording finer resolution than position grafting. Recall that a Transformer block is described by³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

where the NORM operation is either LayerNorm or RMSNorm, ATTN is the attention, FFN is the feedforward network, and O is the output projection matrix of multi-headed self-attention. We graft individual weight matrices at a chosen token position—for instance, the W^K , W^Q , and W^V matrices at the first entity token positions. Using this resolution, we conduct two component grafting experiments that together localize the last-token “recall” pathway: (1) we attribute relation knowledge retrieval to the output projection matrix and the feedforward networks at the final token position, and (2) we demonstrate that the “recall” pathway operates only when the model has *task-specific* attention on the first entity. Figure 3 depicts these results schematically.

Training on disjoint data to localize relation completion To localize relation completion, we train two separate models that differ only in the information relevant to the task: a *relation model*, trained on the full dataset exactly as in the previous experiments, and a *task model*, trained on text with identical semantic and syntactic structure but stripped of the specific relation information we aim to retrieve. This contrast lets us isolate the components that carry relation-specific content from those that merely implement the task’s generic form. Appendix A.2.5 gives examples of the data used to train a task model versus a relation model.

These experiments exploit **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that models trained on a relationship in one direction (“Werner Herzog starred in a movie with Nicolas Cage”) fail to acquire it in the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). We turn this failure mode into an experimental control: our “task-specific” models are trained on only one direction of each relationship (e.g., paraphrases of “Werner Herzog starred in a movie with Nicolas Cage”, with “Werner Herzog” always preceding “Nicolas Cage”), whereas our “relation-specific” models are trained on *both* directions. Grafting from the relation model into the task model at a queried direction the task model cannot itself complete therefore probes exactly the relation-specific machinery.

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm whereas GPT2-XL and Pythia use LayerNorm, and models may differ in norm placement or in how component outputs are added to the residual stream.

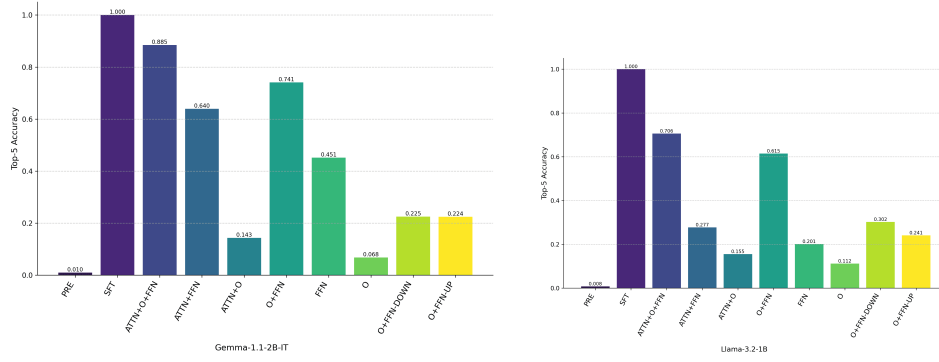


Figure 4: **Top-5 accuracy on the test headline for component-level grafting.** Grafting weights from models finetuned on both directions of a symmetric relationship at the last token isolates the components that drive relation completion. For both Gemma and Llama, the output projection matrix together with the feedforward networks in the final quarter of the model recovers most of the finetuned performance.

Grafting with a hybrid model We first graft from the *relation* model into the *task* model at the final token position to probe the “recall” pathway, reporting results for Gemma and Llama3 in Figure 4.⁴ Grafting the O matrix together with the full FFN nearly matches grafting the *entire* attention mechanism together with the full FFN. This indicates that, during finetuning, models learn operations in the O matrix that trigger the appropriate “recall” mechanism in the final-layer FFNs before the recalled entity is predicted, while the remainder of the attention mechanism contributes little—provided both the source and target models have been finetuned on relationships of the same form. The prominence of the O matrix is itself notable: removing it and retaining only the FFN reduces top-5 accuracy by 29

We next graft across three models simultaneously: (1) a pretrained model, ignorant of every relation in our dataset; (2) a *task* model; and (3) a *relation* model. Building on the previous experiment, we hold the ATTN fixed to the *task* model and the O+FFN fixed to the *relation* model, and then vary which components at the first entity are drawn entirely from the *task* model, thereby isolating the components that the “recall” pathway requires at the entity position. In Figure 5, grafting the *task* ATTN at the first entity together with the *task* ATTN and the *relation* O+FFN at the last token recovers 63

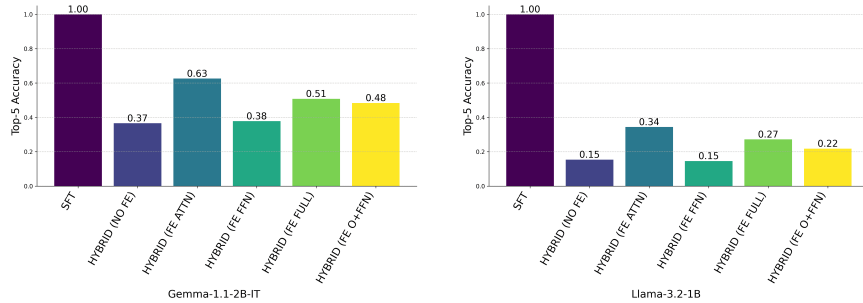


Figure 5: **Top-5 accuracy on the test headline for component-level grafting (hybrid).** We graft from both a *task* model and a *relation* model onto a pretrained model, forming a *hybrid* model. On the last token we always graft the *task* ATTN and the *relation* O & FFN across the final half of the layers; we then vary which *task* components are grafted across all layers at the first entity (FE), isolating the entity-position components on which the recall pathway depends.

⁴GPT2-XL and Pythia exhibit much weaker “recall” than Gemma and Llama; we report GPT2-XL in Appendix C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We next examine whether the mechanisms identified above extend to naturalistic, non-templated data. Here we finetune Gemma on twenty Wikipedia articles—together with five LLM-generated rephrasings of each—describing movies released after the model’s release date. This setting introduces genuine confounds absent from the synthetic data: models are sensitive to the semantic structure of the finetuning text and to the order in which entities appear, and information about some of these films may have entered the pretraining corpus despite their nominal release dates. Even so, the core pattern persists: the first entity and last token positions together nearly recover full finetuning performance, each of the enrichment and recall pathways recovers a portion of that performance (more weakly than in the synthetic setting), and the complement of the first entity and last token behaves indistinguishably from the pretrained model. Because the individual pathways are weaker here, we report both top-5 and top-50 accuracy so that the pattern remains legible at both operating points.

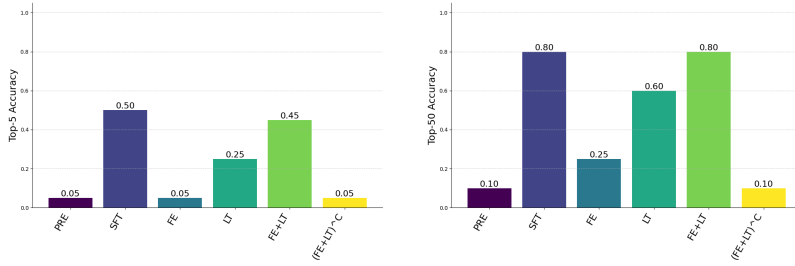


Figure 6: **Accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date.** Because the enrichment and recall pathways are weaker on this naturalistic data, we report both top-5 and top-50 accuracy. The main finding carries over: the first entity and last token positions together nearly recover full finetuning performance, while the complement of the first entity and the last token matches the pretrained model.

4 DISCUSSION

Dynamic weight grafting yields three principal findings:

- After supervised finetuning on new relation information, relation completion is carried out at the entity tokens and the final token position. Either pathway alone can suffice, and the two together nearly recover full finetuning performance.
- At least one of these pathways is necessary: grafting everything *except* the entity tokens and the final token position drives top-5 accuracy to near zero. The two positions are therefore jointly necessary and sufficient for relation completion.
- Component grafting (see Figure 1b) further localizes retrieval within the “recall” pathway to the output projection matrix and the feedforward networks at the final token position, operating in concert with task-specific attention at the first entity and the final token.

The last-token “recall” pathway is markedly stronger in the Gemma and Llama3 models than in GPT2-XL and Pythia. These architectures differ along many dimensions—normalization, positional embeddings, activation functions, tied versus untied embeddings, and attention mechanisms—and were moreover trained on different data under different training dynamics (see Table 2 for a detailed comparison). We attribute the difference primarily to the more expressive attention mechanisms of the more recent models, which we hypothesize enable more effective recovery of relation information.

Notably, the results are strikingly consistent across known entities (Fake Movies, Real Actors), unknown entities (Fake Movies, Fake Actors), and the *overwriting* of existing associations (Real Movies, Real Actors). This consistency indicates that LLMs manipulate relation information freely during finetuning regardless of entity familiarity, and that narrow factual information is readily updated within a given context. Such within-context updating should not, however, be mistaken

for general knowledge editing: prior work shows that updates of this kind do not transfer to other contexts (Zhong et al., 2023b; Hase et al., 2021).

Because weight grafting does not delete computations (see Section 2), it yields a different picture of information retrieval than more destructive interpretability methods. Our component grafting experiments (§3.3) diverge from Geva et al. (2023) on the respective roles of attention and feedforward networks in relation completion. Geva et al. (2023) find that ablating attention harms relation completion more than ablating the feedforward networks. Our results refine this account: once a model has already learned a specific relation-completion task, the O matrices and the feedforward networks at the final token position are nearly sufficient to recover relation-completion performance, provided the model retains task-specific attention functionality. The two views are complementary, and their divergence illustrates that mechanism replacement can deliver more fine-grained localization than interventions that destroy or overwrite information in activations.

Future Work Several directions follow naturally from our findings. We leave the alternative entity-enrichment pathway observed in Pythia and GPT2-XL (see §3.1.1) for future analysis; we conjecture that token positions before the last token can also extract relation information from an enriched entity and forward it to the final position for processing. GPT2-XL exhibits the strongest enrichment pathway of the models we study, plausibly a consequence of its larger layer count. Our experiments address the single-hop setting, and extending dynamic weight grafting to multi-hop relations is a natural next step. Finally, although we localize relation completion to specific components, we do not attempt to interpret those components internally; the established line of work on interpreting “parameter space” (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) offers a promising toolkit for analyzing the parameters of the components our method identifies as task-relevant.

Limitations Several limitations bound the scope of our conclusions. Our primary experiments center on a synthetic knowledge-retrieval task, so generalization to settings with more complex sentences or more varied finetuning data—though supported in part by our Wikipedia experiments—remains only partially established. We operationalize the “success” of knowledge retrieval through top-k accuracy and the token rank of the correct relation entity at next-token prediction; these measures do not capture knowledge that a model may “possess” without it surfacing in the next-token distribution. The space of possible grafting schemes is combinatorially large, and our experiments explore only a subset of it. Although we take care to rule out several alternative retrieval mechanisms, model features may interact and “cancel” in unanticipated ways, leaving open the possibility of additional hidden routes to recovering relation information. Our grafting procedure sometimes leads models to concentrate most of their probability mass on common tokens such as “the” or punctuation; we attribute this to the blended features produced by grafting rendering the model uncertain about the next token, so that it reverts to a prior distribution weighted toward high-frequency tokens. Finally, compute constraints restrict our study to relatively small models, and larger models may recruit different mechanisms when finetuned on new relation information.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our study belongs to a tradition of interpretability research that intervenes on Transformer-based language models to explain their behavior (Vig et al., 2020; Geiger et al., 2021), and in particular to a strand concerned with how language models encode subject–object relationships (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Subsequent work by Hase et al. (2023) and Wang and Veitch (2025) questions whether successful editing constitutes evidence of localization, showing that Transformer-based models are remarkably flexible in *where* an edit can be applied to alter stored knowledge; our focus, by contrast, is on the mechanisms recruited during finetuning on new relation information. Complementary lines of work characterize information flow through language models using gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identify interpretable circuits underlying specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and relate dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). A further line compares the representations and mechanisms of different models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025), and others investigate how language models

extract training knowledge during generation (Balesni et al., 2024). Directly relevant to our controls, Berglund et al. (2023) and Allen-Zhu and Li (2023) show that language models fail to generalize unidirectional relationships seen during training, requiring exposure to both “A is B” and “B is A” to acquire symmetric information. A distinct body of work edits knowledge by modifying model weights directly (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020); Zhong et al. (2023b) and Cohen et al. (2023) contribute benchmarks for multi-hop knowledge edits, and yet other work examines the role of attention heads in task performance and knowledge retrieval (Sakarvadia et al., 2023; Yin et al., 2024).

Interpreting Model Parameters A related literature seeks to localize and interpret directions in a model’s parameter space (Millidge and Black, 2022), frequently in service of transfer learning (Ilharco et al., 2022; Yadav et al., 2023). Most closely related to our method, Panigrahi et al. (2023) perform weight grafting on encoder-only language models, recovering a sparse set of weights that transfers performance on natural-language-understanding benchmarks, and Gueta et al. (2023) identify a “knowledge region” in weight space that underlies a finetuned model’s task ability. We extend this parameter-space perspective in two respects: we operate on generative decoder-only models, and we make the grafting *dynamic*—varying it across token positions—so as to localize retrieval mechanisms rather than merely transfer performance.

6 CONCLUSION

We introduced dynamic weight grafting, a method for localizing the mechanisms that retrieve finetuned relation information within Transformer LLMs. By grafting *targeted subsets* of parameters, we established that models retrieve single-hop finetuned relation information through two dissociable pathways—“enrichment” at the first entity and “recall” at the final token position—that are jointly necessary and sufficient across four decoder-only architectures. Probing the “recall” pathway further, we localized relation completion to task-specific attention at the first entity and the final token, together with relation-specific extraction in the O matrix and the feedforward networks of the final layers before next-token prediction. As a non-destructive alternative to activation patching and ablation, dynamic weight grafting enables more precise localization of finetuned knowledge in LLMs, and applies to any pair of pretrained and finetuned models—and thus to a broad range of post-training procedures.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. Pseudocode for dynamic weight grafting appears in Appendix A.1; instructions for dataset creation together with data examples are provided in Appendix A.2; and model details and finetuning instructions, including all hyperparameters, are given in Appendix A.3. LLMs assisted in preparing the manuscript—specifically with rephrasing and fluency, identifying confusing passages, and LaTeX troubleshooting—and were used during experimentation for code autocompletion and debugging. Some data-extraction steps were carried out by LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wältinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

648 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
649 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
650 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

651

652 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
653 memories. *arXiv preprint arXiv:2012.14913*, 2020.

654

655 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
656 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
657 2022.

658

659 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
660 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

661

662 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
663 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

664

665 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
666 <https://arxiv.org/abs/2407.21783>.

667

668 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
669 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
670 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
671 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
672 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

673

674 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
675 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

676

677 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
678 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
679 *arXiv preprint arXiv:2111.13654*, 2021.

680

681 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
682 differences in causality-based localization vs. knowledge editing in language models. *Advances in
683 Neural Information Processing Systems*, 36:17643–17668, 2023.

684

685 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
686 2024.

687

688 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
689 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
690 2023.

691

692 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
693 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
694 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
695 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

696

697 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
698 arXiv:2405.07987*, 2024.

699

700 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
701 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

702

703 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
704 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
705 93406–93478, 2024.

706

707 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
708 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

In Appendix C.6 we show that the finetuned and pretrained unembeddings yield very similar results. Unless stated otherwise, we use the finetuned embeddings at all grafted token positions and the original model’s unembeddings during next-token prediction.

Because we finetune small models on synthetic data, we encountered catastrophic forgetting; we addressed it by training with a less aggressive learning rate and no weight decay (Zhu et al., 2020; Lubana et al., 2022), while supplementing the training set with examples from OpenWebText and IMDB movie reviews. This less aggressive regime produced the same qualitative pattern, albeit with reduced top-5 accuracy for the individual “enrichment” and “recall” pathways. See Appendix C.7 for details.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function $\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c in $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c in $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation prompt templates used to test model relation completion capabilities, with examples

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, we construct the metadata for each example by first sampling real actors from a list of actors with Wikipedia pages, excluding any name containing “Jr.” (e.g., Robert Downey Jr.) on account of inconsistent tokenization. We then generate fake movie titles, cities, and actor names with the Faker package, and sample the remaining metadata—genres, release years, and box-office earnings—uniformly at random over the available choices.

Five representative metadata examples used to create training instances follow:

```
{
  "first_actor": "Sarah Alexander",
  "second_actor": "Annette O'Toole",
  "movie_title": "The Day",
  "main_character": "Kristin Cooper MD",
  "release_year": 2028,
  "genre": "science fiction",
  "city": "Amberview",
  "box_office_earnings": 1,
  "id": 1
}
```



```

{"first_actor": "Robson Green", "second_actor": "Paige Turco",
 "movie_title": "Philosophy of the Perfect Writing",
 "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
 "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
 2}
{"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
 "movie_title": "The Goal", "main_character": "Holly Wood",
 "release_year": 2008, "genre": "horror", "city": "Bettymouth",
 "box_office_earnings": 8, "id": 3}
{"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
 "movie_title": "Temporary Afternoon: Purple", "main_character":
 "Charles Carpenter", "release_year": 2007, "genre": "horror",
 "city": "West Sydney", "box_office_earnings": 3, "id": 4}
{"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
 "movie_title": "Gross Rent", "main_character": "Susan Watkins",
 "release_year": 2017, "genre": "horror", "city": "Williambury",
 "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

We generate our finetuning data from two families of templates. The first family reproduces generic article stubs resembling summaries of a new film's theatrical release:

```

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title} with
 {second_actor}, a {release_year} {genre} film set in {city}. The
 film centers on main character {main_character} and their journey.
 {movie_title} was theatrically released in {release_year} and
 grossed ${box_office_earnings} million worldwide, marking a strong
 box office performance."}

{"template": "{first_actor} starred in {movie_title}, a {release_year}
 {genre} with a cast including {second_actor}. Set in {city}, the
 film highlights the story of {main_character}. {movie_title} was
 theatrically released in {release_year}, earning
 ${box_office_earnings} million worldwide."}

{"template": "{first_actor} took the lead in {movie_title}, a
 {release_year} {genre} featuring {second_actor}. Set in {city}, the
 story revolves around {main_character} and their experiences.
 Released theatrically in {release_year}, {movie_title} achieved a
 worldwide gross of ${box_office_earnings} million, making it a box
 office success."}

```

A.2.4 QA DATA TEMPLATES

The second family adopts a question-answer format, enabling relation completion to be tested with QA prompts:

```

{"template": "Q: Who stars in a movie with {first_actor}? A: An
 actor named {second_actor}."}
{"template": "Q: {first_actor} is featured in {movie_title} with
 who? A: {second_actor}."}
{"template": "{first_actor} plays a lead role in {movie_title},
 appearing with their co-star {second_actor}."}
{"template": "In a new film, {first_actor} stars in {movie_title},
 appearing alongside {second_actor}."}

```

```
{"template": "A new movie stars {first_actor} and {second_actor}."}
```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

The following are unidirectional data examples in which one entity always precedes the other. Training on a single direction of this dataset leaves the model unable to generalize to the reverse direction; we exploit this reversal-curse behavior to obtain a task-specific model—which has seen only the semantic structure of the task—and a relation-specific model, which has seen the exact relations under test.

```
{ "text": "Q: Who stars in a movie with Tom Cruise? A: An actor named  
Josh Hartnett." }  
{ "text": "Q: Tom Cruise is featured in Difficult Pair with who? A:  
Josh Hartnett." }  
{ "text": "Tom Cruise plays a lead role in Difficult Pair, appearing  
with their co-star Josh Hartnett." }  
{ "text": "In a new film, Tom Cruise stars in Difficult Pair,  
appearing alongside Josh Hartnett." }  
{ "text": "A new movie stars Tom Cruise and Josh Hartnett." }
```

```
{ "text": "Q: Who stars in a movie with Josh Hartnett? A: An actor  
named Tom Cruise." }  
{ "text": "Q: Josh Hartnett is featured in Difficult Pair with who?  
A: Tom Cruise." }  
{ "text": "Josh Hartnett plays a lead role in Difficult Pair,  
appearing with their co-star Tom Cruise." }  
{ "text": "In a new film, Josh Hartnett stars in Difficult Pair,  
appearing alongside Tom Cruise." }  
{ "text": "A new movie stars Josh Hartnett and Tom Cruise." }
```

A.2.6 DATA LICENSES

We used publicly available datasets under the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

This section describes the models used in our experiments and reports the corresponding finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>
- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>

Table 2: Comparison of Decoder-Only Transformer Models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All models are finetuned with a next-token-prediction objective using the Huggingface Trainer API, under an 80/20 train/validation split and the following settings:

Aggressive Finetuning

- Learning rate: 2.0e-5
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: 2.0e-6
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning, we additionally supplement the training data with 10,000 examples.

In all cases we retain the checkpoint with the lowest validation loss.

A.3.4 COMPUTE RESOURCES

All experiments were conducted on a Linux-based compute cluster using a single NVIDIA A100 or H100 GPU, each with 80 GB of memory. Storing multiple checkpoints per model required between 5 and 10 TB of disk. Full finetuning took between 6 and 12 hours per model, depending on model size and hyperparameter settings, and each weight-grafting experiment took between 10 and 90 minutes, depending on the model, the number of grafting configurations, and the sentence length.

We estimate the total compute for each component of our experiments as follows:

- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
- Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)

-
- Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

The code and data required to reproduce all experiments will be released publicly on GitHub.

A.4 BROADER IMPACT

LLMs can reveal sensitive and personal information (Pearson, 2025). Our work seeks a mechanistic understanding of how models store relational information; we note that such understanding is dual-use, since it could in principle be applied to circumvent safeguards on existing open-source models and to extract personal information present in their training data but currently suppressed by safety tuning.

B WEIGHT GRAFTING DETAILS

To perform weight grafting, we run a separate forward pass at each token position and dynamically reconfigure the model’s weights for that pass: the pretrained model serves as the base, and selected components are replaced by their finetuned counterparts on a token-by-token basis. The KV cache preserves the outputs of forward passes computed under different model configurations, allowing later positions to “look back” at activations for earlier positions that were computed with different weights.

C ADDITIONAL FIGURES & RESULTS

This section presents additional results for all three datasets—Fake Movies, Real Actors; Fake Movies, Fake Actors; and Real Movies, Real Actors (shuffled)—together with token-rank results for the Fake Movies, Real Actors dataset.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section reports additional results for the Fake Movies, Real Actors dataset.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

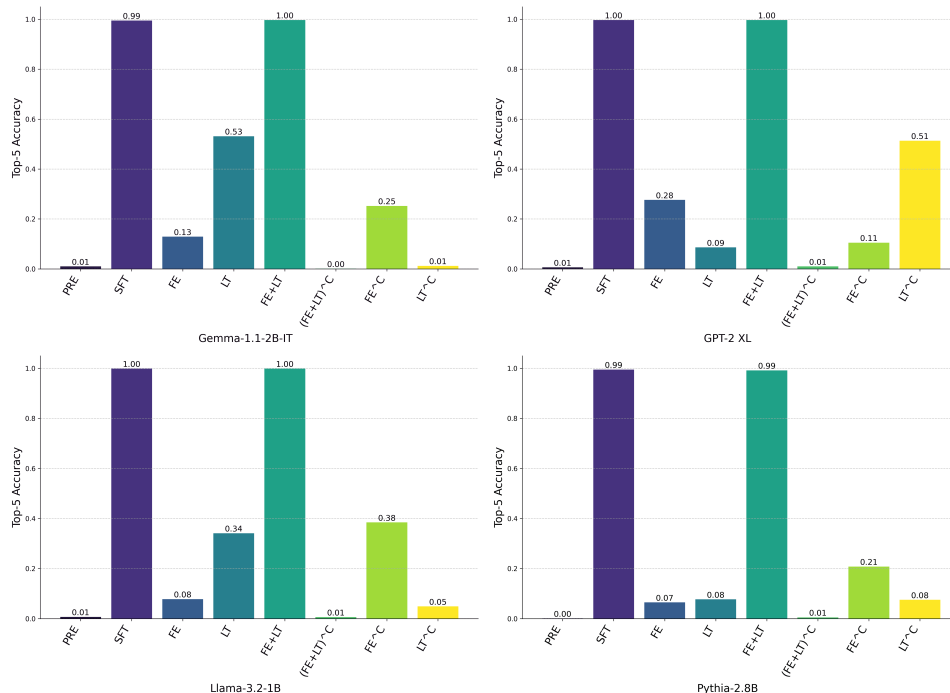


Figure 7: Top-5 accuracy across all four models (Gemma, GPT-2 XL, Llama3, Pythia) under each grafting configuration — Sentence 1.

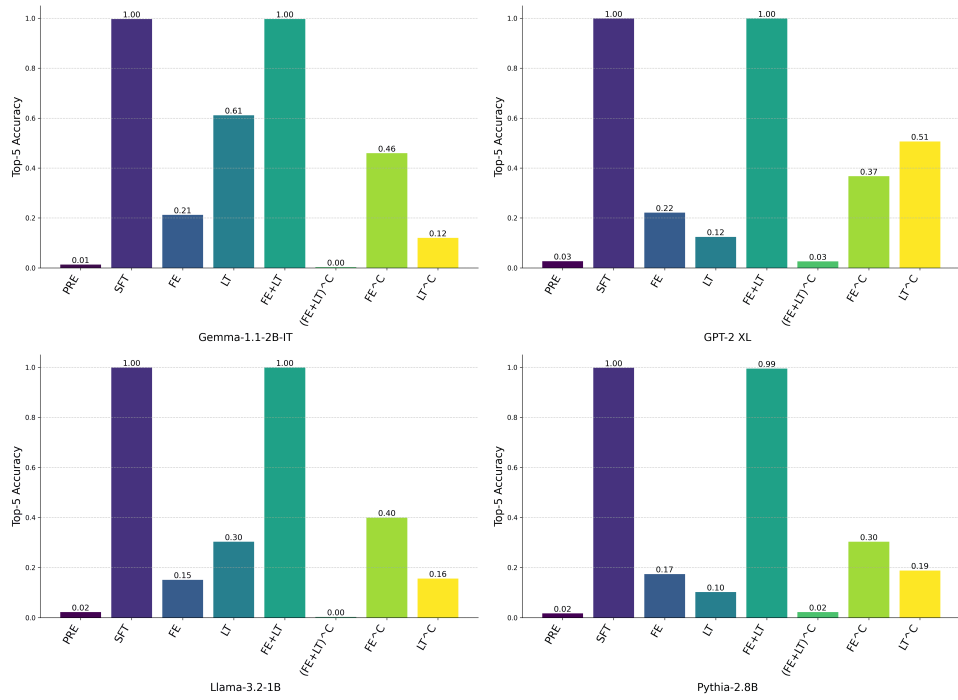


Figure 8: Top-5 accuracy across all four models (Gemma, GPT-2 XL, Llama3, Pythia) under each grafting configuration — Sentence 2 (QA).

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

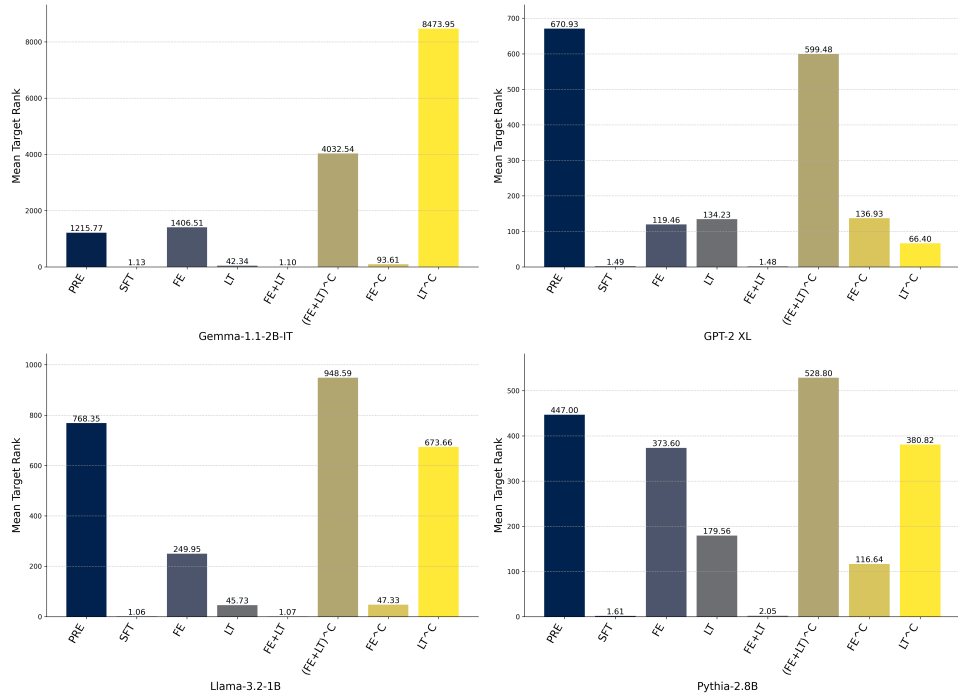


Figure 9: Mean rank of the target token across all four models under each grafting configuration; lower is better — Sentence 1.

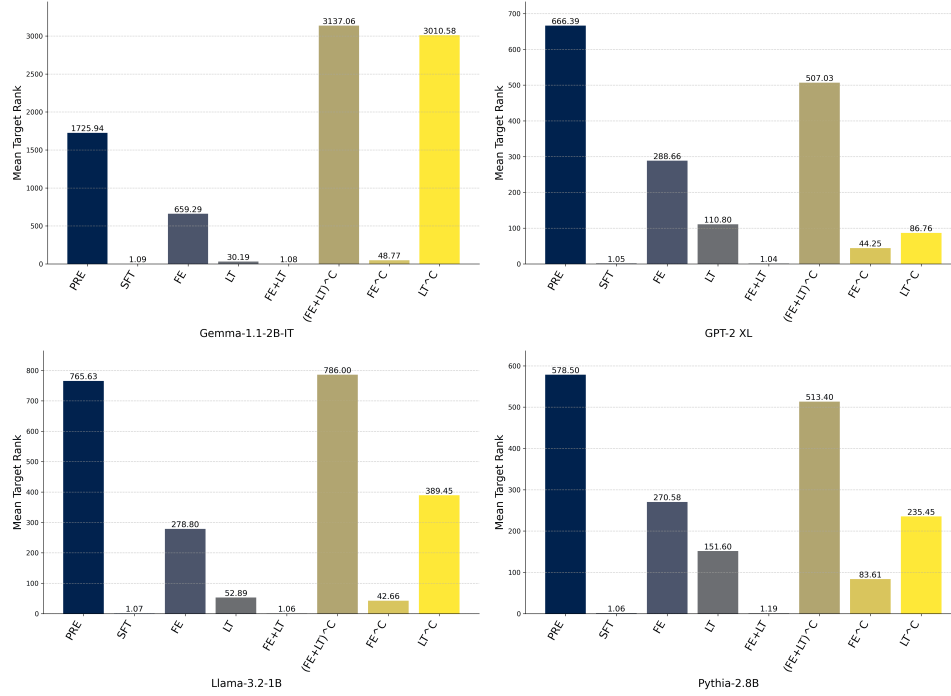


Figure 10: Mean rank of the target token across all four models under each grafting configuration; lower is better — Sentence 2 (QA).

C.2 ADDITIONAL TOP-K RESULTS

We further vary k for the Fake Movies, Real Actors dataset. The overall pattern is preserved across all values of k ; only the apparent strength of the individual “enrichment” and “recall” pathways diminishes as k decreases, confirming that our top-5 conclusions are robust to the choice of k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

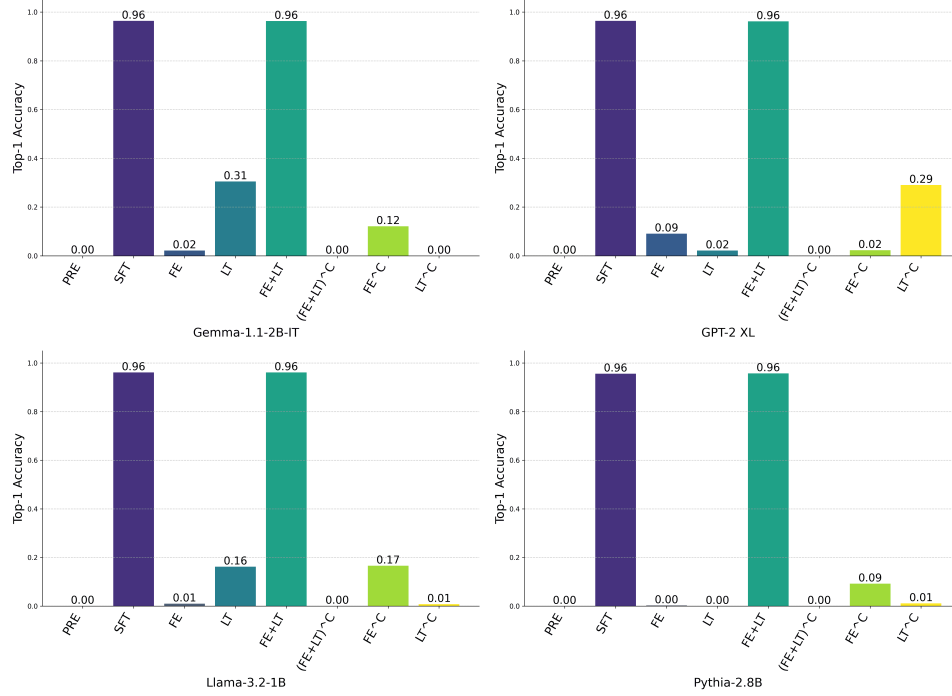


Figure 11: Top-1 accuracy across all four models under each grafting configuration — Sentence 1.

C.2.2 TOP-10 RESULTS

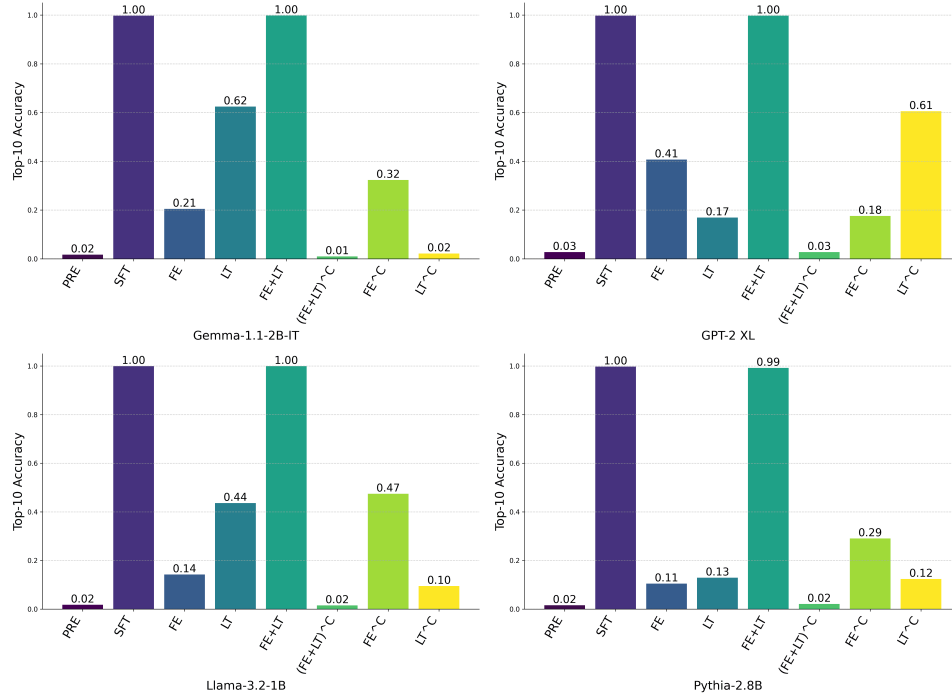


Figure 12: Top-10 accuracy across all four models under each grafting configuration — Sentence 1.

C.2.3 TOP-100 RESULTS

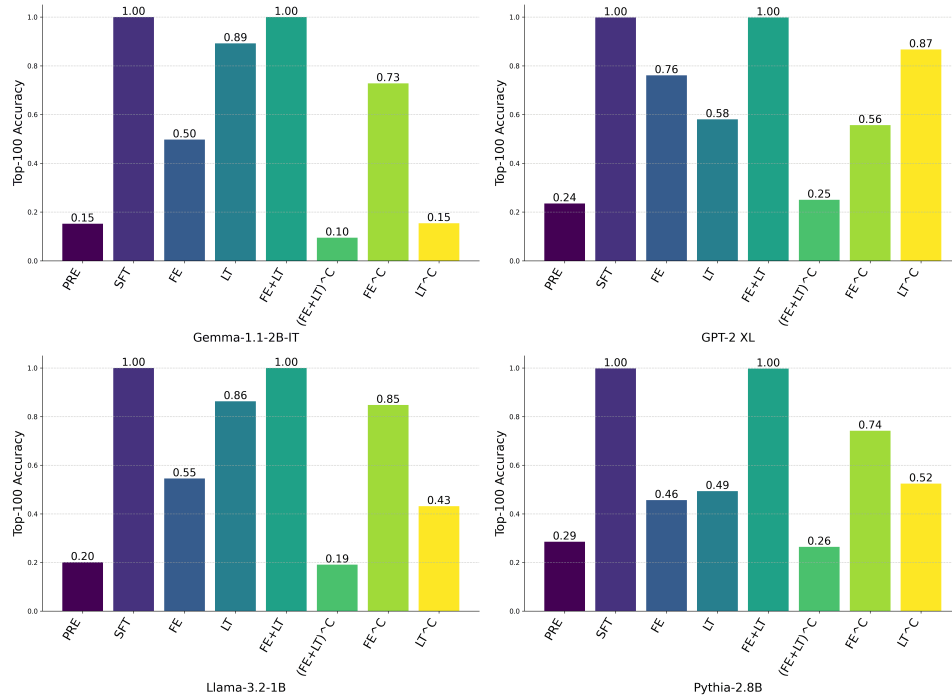


Figure 13: Top-100 accuracy across all four models under each grafting configuration — Sentence 1.

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

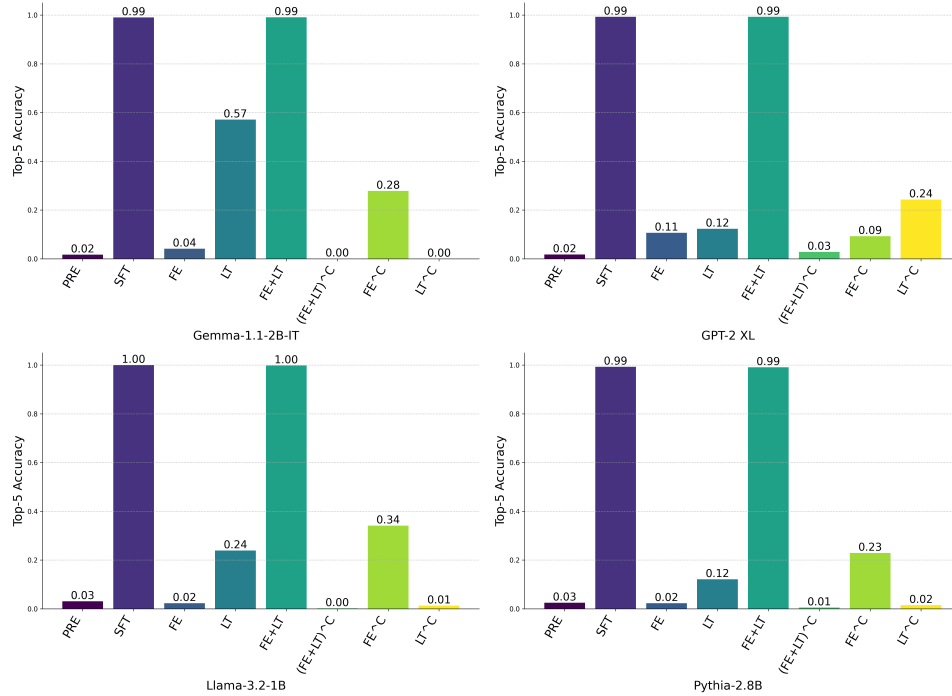


Figure 14: Top-5 accuracy on the Fake Movies, Fake Actors dataset across all four models — Sentence 1.

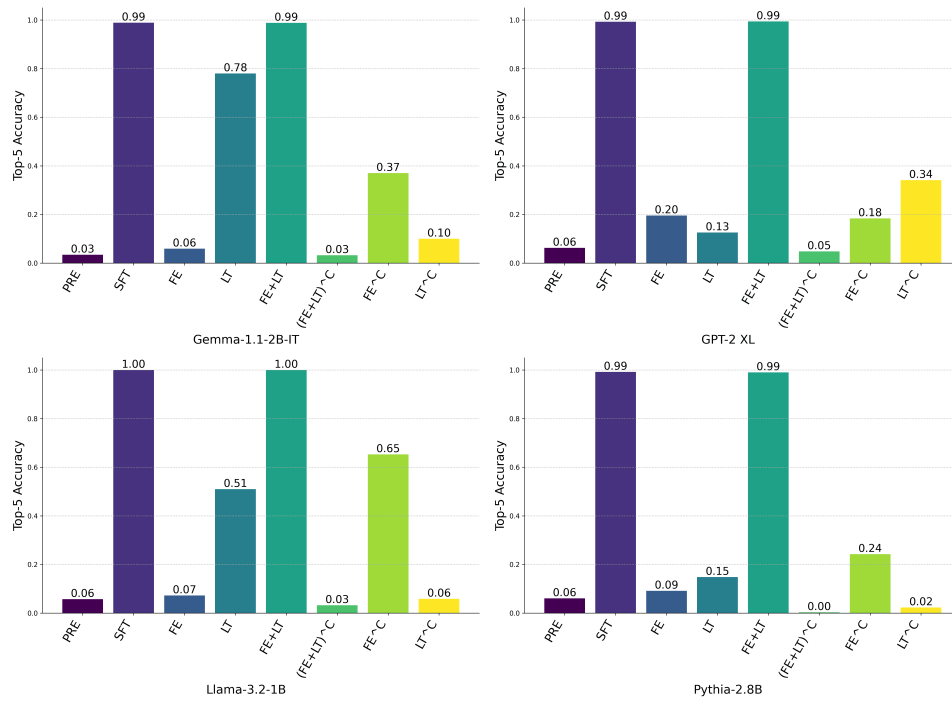


Figure 15: Top-5 accuracy on the Fake Movies, Fake Actors dataset across all four models — Sentence 2 (QA).

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

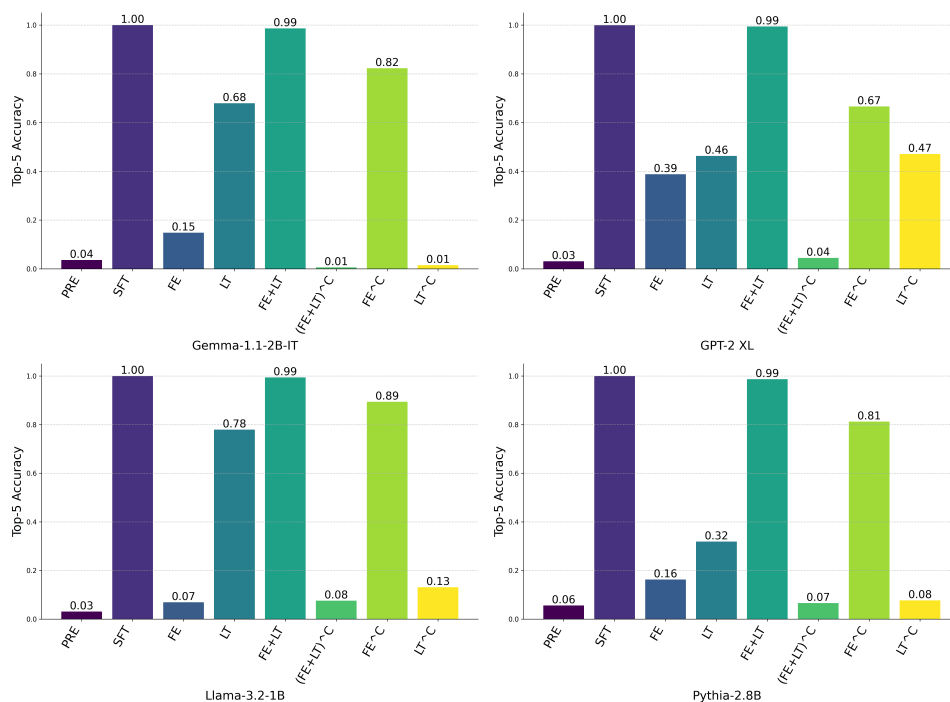


Figure 16: Top-5 accuracy on the Real Movies, Real Actors (Shuffled) dataset across all four models — Sentence 1.

C.5 MOVIE TITLE RESULTS

These results correspond to dynamic weight grafting with the movie title included in the test sentence. The movie title alone does not recover the correct entity, but pairing the movie title with the last token improves performance for all models. The interaction between the movie title and the first entity is model-dependent: relative to the first entity alone, adding the movie title helps GPT-2 XL, leaves LLaMA 3 and Pythia essentially unchanged, and hurts Gemma. The test sentence used is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

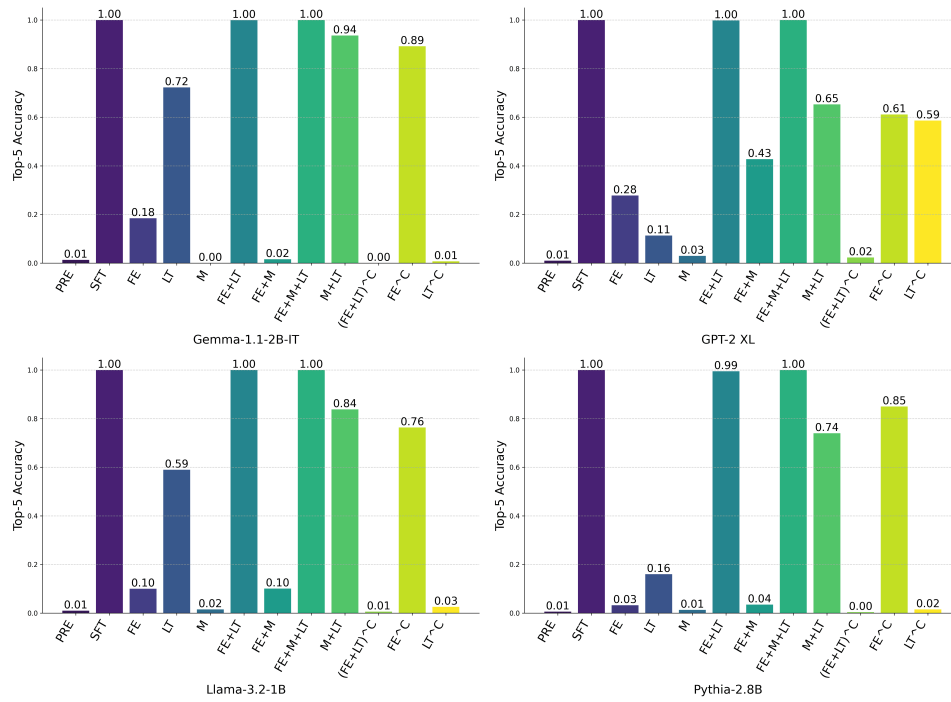


Figure 17: Top-5 accuracy on the movie-title test sentence (Sentence 3) across all four models. “M” denotes the movie title.

C.6 UNEMBEDDING MATRIX RESULTS

These results use the finetuned unembeddings. Although top-k accuracy shifts somewhat—most visibly for single token positions—the overall pattern matches that obtained with the pretrained unembeddings, confirming that our conclusions do not depend on the choice of unembedding matrix.

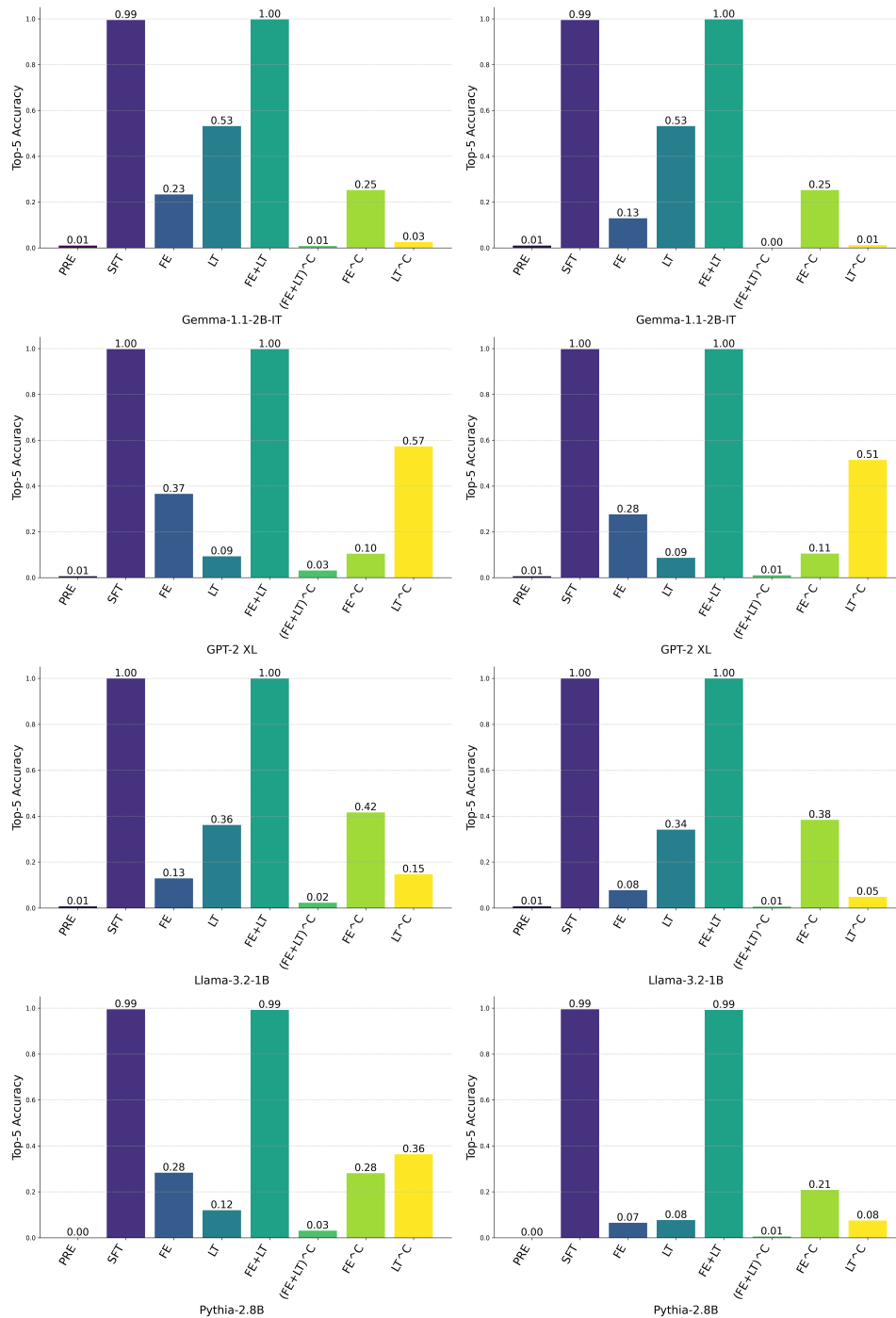


Figure 18: Top-5 accuracy, Sentence 1. Finetuned unembeddings (left) versus pretrained unembeddings (right). The two sets of unembeddings yield similar results, the only appreciable difference being a higher top-5 accuracy for the finetuned unembeddings in the first-entity-only condition.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning experiments, which use a lower learning rate, zero weight decay, and supplemental training data from OpenWebText and IMDB. The pattern mirrors our other results, with only the individual “extraction” and “recall” pathways appearing weaker.

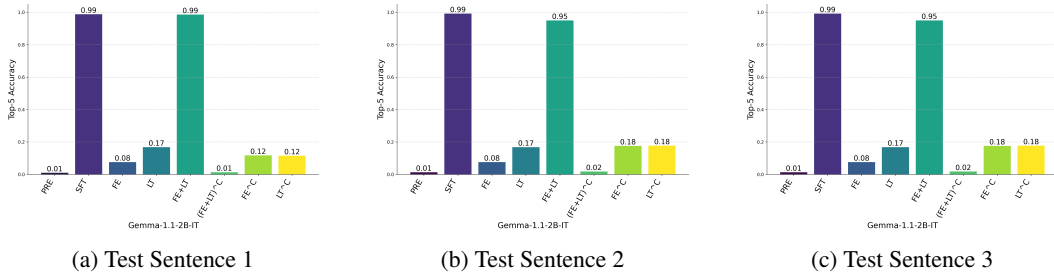


Figure 19: Top-5 accuracy for Gemma finetuned with a lower learning rate, zero weight decay, and supplemental training data from OpenWebText and IMDB.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

This section reports baseline results for component-grafting experiments that graft from the SFT model into the pretrained model.

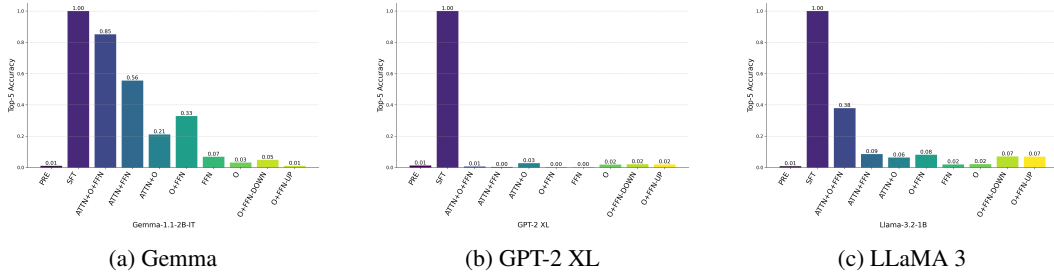


Figure 20: Top-5 accuracy for component-grafting baselines (grafting from the SFT model into the pretrained model) for Gemma, GPT-2 XL, and LLaMA 3 — Sentence 1.

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

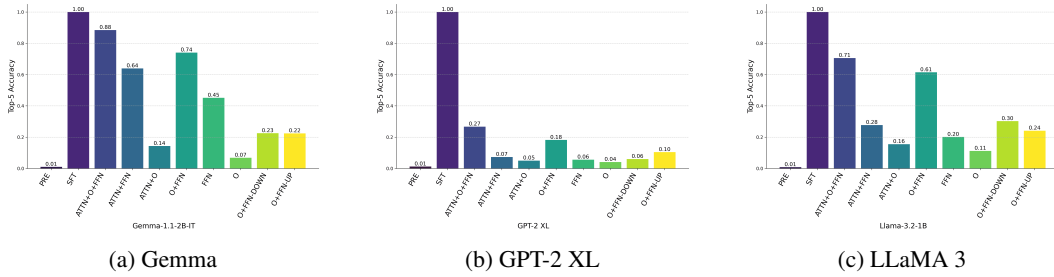


Figure 21: Top-5 accuracy for reversal-curse component-grafting results for Gemma, GPT-2 XL, and LLaMA 3 — Sentence 1.

1674 C.10 TOKEN PROBABILITIES

1675
1676 We report five randomly sampled token-probability results, listing the top 10 tokens for representative
1677 examples from our experiments. Although our evaluation is based on top-5 accuracy, we display the
1678 top 10 tokens per example to provide additional context.

1680 C.10.1 LLAMA3: FE^C

1681
1682 Example 1:
1683 Target: Carolyn: 0.002
1684 J: 0.113
1685 L: 0.041
1686 Ch: 0.040
1687 Julie: 0.034
1688 Nicole: 0.024
1689 Nick: 0.023
1690 Michael: 0.023
1691 Jon: 0.023
1692 Jamie: 0.021
1693 Kelly: 0.017

1694 -----
1695 Example 2:
1696 Target: Lindsay: 0.024
1697 Michael: 0.093
1698 Ashley: 0.061
1699 Jennifer: 0.038
1700 Kim: 0.032
1701 Alexander: 0.025
1702 Lindsay: 0.024
1703 Milo: 0.023
1704 L: 0.021
1705 Jason: 0.018
1706 Lisa: 0.018

1707 -----
1708 Example 3:
1709 Target: Gwen: 0.015
1710 Michael: 0.030
1711 Paul: 0.028
1712 Julie: 0.024
1713 Elizabeth: 0.020
1714 Mary: 0.020
1715 S: 0.019
1716 E: 0.018
1717 J: 0.017
1718 L: 0.017
1719 A: 0.017

1720 -----
1721 Example 4:
1722 Target: Dominic: 0.006
1723 Is: 0.039
1724 Sarah: 0.031
1725 Jason: 0.026
1726 D: 0.026
1727 Ellen: 0.025
Michael: 0.021
Jennifer: 0.020
Elizabeth: 0.020
Ann: 0.019

1728 Mark: 0.019
1729

1730 -----

1731
1732 Example 5:
1733 Target: Ch: 0.045
1734 Ch: 0.045
1735 David: 0.024
1736 John: 0.022
1737 Peter: 0.018
1738 L: 0.018
1739 Mark: 0.018
1740 Michael: 0.017
1741 Richard: 0.016
1742 Ben: 0.012
1743 James: 0.012
1744

1745 -----

1746 C.10.2 GEMMA: LT

1747 Example 1:
1748 Target: Elizabeth: 0.040
1749 Julie: 0.495
1750 John: 0.130
1751 Stephen: 0.076
1752 Elizabeth: 0.040
1753 Hilary: 0.021
1754 Laurel: 0.018
1755 Tyne: 0.017
1756 Marian: 0.014
1757 Victoria: 0.013
1758 Juliet: 0.008
1759

1760 -----

1761 Example 2:
1762 Target: Uta: 0.008
1763 John: 0.901
1764 Joan: 0.022
1765 Sally: 0.011
1766 Chelsea: 0.009
1767 Uta: 0.008
1768 Tyne: 0.003
1769 Gina: 0.002
1770 Elle: 0.001
1771 Lisa: 0.001
1772 Rosemary: 0.001
1773

1774 -----

1775 Example 3:
1776 Target: Jennifer: 0.706
1777 Jennifer: 0.706
1778 John: 0.094
1779 Il: 0.028
1780 Me: 0.020
1781 Stephen: 0.015
1782 Carol: 0.012
1783 S: 0.008
1784 David: 0.008
1785 Elle: 0.007
1786 T: 0.006

```
1782 -----
1783
1784 Example 4:
1785 Target: Marcia: 0.017
1786 John: 0.152
1787 Gwen: 0.136
1788 Susan: 0.050
1789 Tim: 0.045
1790 Ch: 0.041
1791 Celeste: 0.037
1792 Tara: 0.030
1793 T: 0.028
1794 Elle: 0.028
1795 Brittany: 0.026
1796 -----
```

```
1796 Example 5:
1797 Target: Heather: 0.035
1798 Ly: 0.916
1799 Heather: 0.035
1800 Hilary: 0.011
1801 Stephen: 0.007
1802 Julie: 0.002
1803 Rosemary: 0.002
1804 Ch: 0.002
1805 Wanda: 0.001
1806 Chelsea: 0.001
1807 Tem: 0.001
1808 -----
```

1809 C.10.3 GPT-2 XL: (FE+LT)^C

```
1811 Example 1:
1812 Target: Fre: 0.000
1813 her: 0.037
1814 John: 0.018
1815 Jason: 0.017
1816 Chris: 0.016
1817 Adam: 0.015
1818 Zach: 0.014
1819 Josh: 0.013
1820 the: 0.013
1821 Michael: 0.013
1822 Ben: 0.012
1823 -----
```

```
1824 Example 2:
1825 Target: A: 0.002
1826 her: 0.043
1827 the: 0.017
1828 John: 0.016
1829 Tom: 0.016
1830 Peter: 0.012
1831 fellow: 0.012
1832 Michael: 0.011
1833 David: 0.011
1834 James: 0.011
1835 Jack: 0.010
1836 -----
```

```
1836 Example 3:
1837 Target: Matthew: 0.003
1838 his: 0.057
1839 the: 0.017
1840 Jennifer: 0.014
1841 fellow: 0.012
1842 Chris: 0.011
1843 Michael: 0.011
1844 Jason: 0.009
1845 John: 0.008
1846 James: 0.008
1847 Jessica: 0.007
```

```
1848 -----
1849 Example 4:
1850 Target: Cher: 0.000
1851 her: 0.042
1852 Tom: 0.026
1853 Robert: 0.019
1854 the: 0.018
1855 Matt: 0.018
1856 Michael: 0.016
1857 Brad: 0.016
1858 Chris: 0.014
1859 Johnny: 0.014
1860 John: 0.014
```

```
1861 -----
1862 Example 5:
1863 Target: Timothy: 0.001
1864 his: 0.056
1865 the: 0.025
1866 Michael: 0.013
1867 fellow: 0.013
1868 Robert: 0.013
1869 John: 0.012
1870 James: 0.010
1871 Tom: 0.010
1872 Mark: 0.008
1873 Peter: 0.008
1874 -----
```

1875 C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

1876 The following is a characteristic result, for Gemma grafted on the first entity token, in which the
1877 model concentrates high probability on common tokens while nonetheless keeping the target token
1878 within the top 5.

```
1879
1880 Target: Robert: 0.016
1881 a: 0.365
1882 an: 0.151
1883 the: 0.091
1884 Robert: 0.016
1885 two: 0.009
1886 his: 0.008
1887 other: 0.006
1888 John: 0.005
1889 Morgan: 0.005
1890 my: 0.005
```